

Assignment 2: Factory Method and Abstract Factory

Astana IT University | School of Software Engineering

Course: ShP-2216 Software Design Patterns | Academic Year: 2026–2027 (Year 2, Trimester 4)

Student: Nursultan Maratov	Instructor: Yerassyl Bekenov
Group: SE-2523	Format: Java 17 Console Application
GitHub Repository: https://github.com/Jozephjostar/logistics-assignment	Submitted Commit: fc38cfc

1. Introduction

This project combines two creational design patterns in a single Java 17 application: **Factory Method** for logistics transport and **Abstract Factory** for cross-platform UI components.

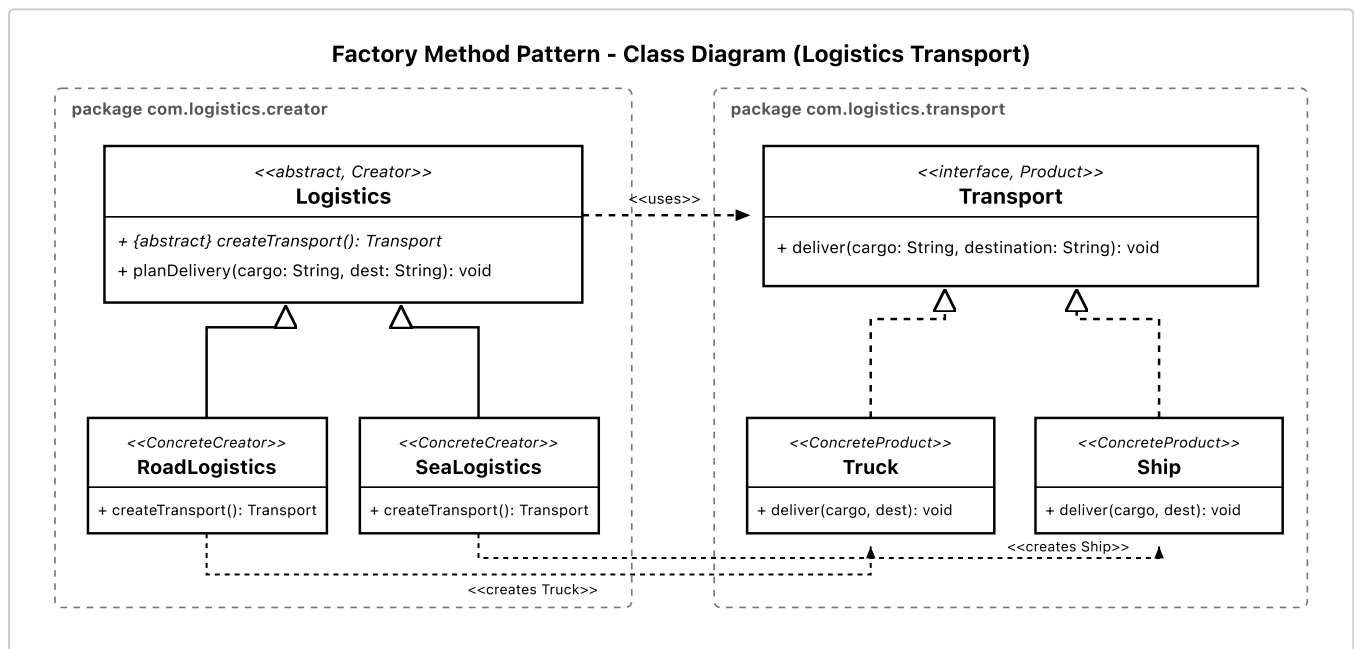
Why Factory Method for Logistics: Logistics operations share a common delivery workflow: planning cargo, creating a transport, and delivering items to a destination. The `Logistics` class defines the overall delivery workflow in `planDelivery()`, while creator subclasses (`RoadLogistics` and `SeaLogistics`) decide which concrete transport (`Truck` or `Ship`) to instantiate. This follows the Open-Closed Principle because new transports can be added without changing the delivery workflow.

Why Abstract Factory for UI Components: A graphical interface requires matching sets of components. A button and checkbox on Windows should look like Windows controls, while macOS components should match macOS styling. The `GUIFactory` interface ensures that `WindowsFactory` creates only Windows controls and `MacOSFactory` creates only macOS controls. The client application (`DeliveryApplication`) uses these components through abstract interfaces, avoiding any hardcoded dependencies or type casting.

2. UML Class Diagrams

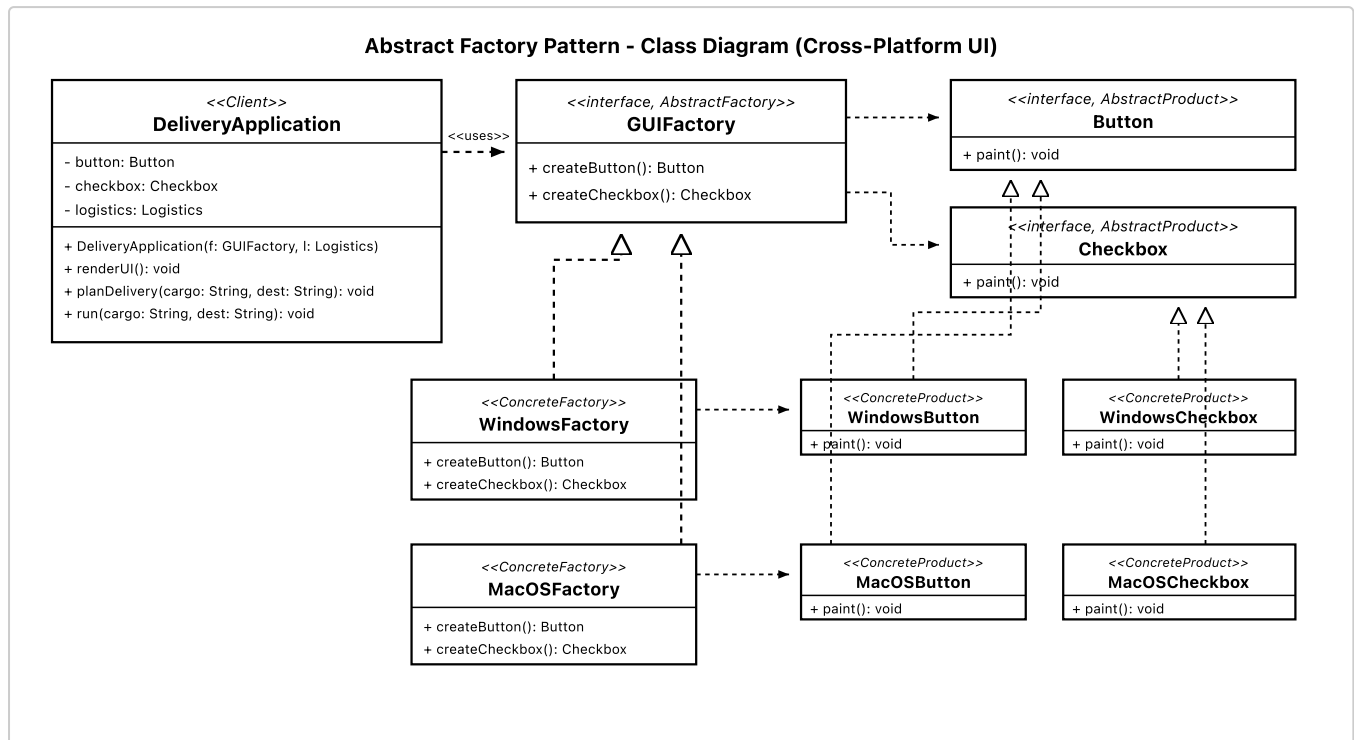
2.1 Factory Method Pattern

The creator hierarchy (`Logistics`) is separated from the product hierarchy (`Transport`). The template method `planDelivery()` accesses products through `createTransport()`.



2.2 Abstract Factory Pattern

The client (`DeliveryApplication`) depends on the abstract factory (`GUIFactory`) and product interfaces (`Button`, `Checkbox`). Concrete factories produce matching component pairs.



3. Clean Code Evidence (Section 7)

1. Meaningful Names

```
public class RoadLogistics extends Logistics {
    @Override
    public Transport createTransport() {
        return new Truck();
    }
}
```

Benefit: Class and method names clearly reflect their roles. `RoadLogistics` clearly indicates a concrete creator for road freight, and `createTransport()` clearly declares the factory method responsibility, avoiding vague names like `Manager` or `Processor`.

2. Small Methods (Single Responsibility)

```
public void renderUI() {
    button.paint();
    checkbox.paint();
}

public void planDelivery(String cargo, String destination) {
    logistics.planDelivery(cargo, destination);
}
```

Benefit: Each method does one specific job. UI rendering is separated from delivery workflow execution. This makes the code easy to read, test, and maintain without unintended side effects.

3. Avoid Duplicated Logic (DRY Principle)

```
public void planDelivery(String cargo, String destination) {
    Objects.requireNonNull(cargo, "Cargo cannot be null");
    Objects.requireNonNull(destination, "Destination cannot be null");

    Transport transport = createTransport();
    if (transport == null) {
        throw new IllegalStateException("Factory method returned null transport");
    }
}
```

```
    transport.deliver(cargo, destination);  
}
```

Benefit: The delivery process (argument validation, transport creation, and invoking delivery) is written once in the abstract `Logistics` class. Subclasses only specify which transport to instantiate, eliminating code duplication.

4. Data Abstraction (Clean Code, Chapter 6)

```
public class DeliveryApplication {
    private final Button button;
    private final Checkbox checkbox;
    private final Logistics logistics;

    public DeliveryApplication(GUIFactory factory, Logistics logistics) {
        this.button = factory.createButton();
        this.checkbox = factory.createCheckbox();
        this.logistics = logistics;
    }
}
```

Benefit: According to Clean Code Chapter 6, abstractions hide implementation details behind interfaces. The client relies exclusively on `Button`, `Checkbox`, `GUIFactory`, and `Logistics`. It does not know or care whether the platform is Windows or macOS, or whether transport is by road or sea.

5. Objects and Encapsulation (Clean Code, Chapter 6)

```
public class Truck implements Transport {
    @Override
    public void deliver(String cargo, String destination) {
        Objects.requireNonNull(cargo, "Cargo must not be null");
        Objects.requireNonNull(destination, "Destination must not be null");
        System.out.println("Truck delivers " + cargo + " to " + destination);
    }
}
```

Benefit: Clean Code Chapter 6 explains that true objects hide their data and expose behavior through methods. The transport products do not expose internal data structures through unnecessary getters and setters; they expose meaningful actions via `deliver()`.

4. Verification Evidence (Section 6)

All required runtime combinations and validation cases were executed and verified:

Check	Input	Expected Result	Actual Output	Status
1	ROAD + WINDOWS	Truck delivery; Windows button & checkbox	Rendering Windows button Rendering Windows checkbox Truck delivers laboratory equipment to Aktau warehouse	PASS
2	SEA + WINDOWS	Ship delivery; Windows button & checkbox	Rendering Windows button Rendering Windows checkbox Ship delivers laboratory equipment to Aktau warehouse	PASS
3	ROAD + MACOS	Truck delivery; macOS button & checkbox	Rendering macOS button Rendering macOS checkbox Truck delivers laboratory equipment to Aktau warehouse	PASS
4	SEA + MACOS	Ship delivery; macOS button & checkbox	Rendering macOS button Rendering macOS checkbox Ship delivers laboratory equipment to Aktau warehouse	PASS
5	AIR + WINDOWS	Clear validation message; no delivery executed	Validation Error: Unsupported delivery mode 'AIR'. Supported modes: ROAD, SEA.	PASS
6	ROAD + LINUX	Clear validation message; no UI constructed	Validation Error: Unsupported UI platform 'LINUX'. Supported platforms: WINDOWS, MACOS.	PASS

Check	Input	Expected Result	Actual Output	Status
7	Missing input	Clear validation message; clean stop	Validation Error: Missing required configuration arguments.	PASS

JUnit 5 Automated Test Execution:

Results: Tests run: 27, Failures: 0, Errors: 0, Skipped: 0 | BUILD SUCCESS

5. Conclusion and Design Reflection

5.1 Pattern Comparison

- **Simple Factory:** A single class with conditional logic (if/switch). It centralizes creation but violates the Open-Closed Principle whenever a new product is added.
- **Factory Method:** Uses inheritance. An abstract creator delegates instantiation to subclasses, making it easy to add new products without modifying existing code.
- **Abstract Factory:** Uses object composition. An interface creates families of related products, guaranteeing that components match without coupling the client to concrete classes.

5.2 Design Reflection: System Extensibility

1. **Adding a new Transport (e.g. Air):** Create `Plane` implements `Transport` and `AirLogistics` extends `Logistics`. Add `AIR` to the enum and startup helper. Existing transport classes, creators, and `DeliveryApplication` remain completely unchanged.
2. **Adding a new UI Family (e.g. Linux):** Create `LinuxButton`, `LinuxCheckbox`, and `LinuxFactory` implements `GUIFactory`. Add `LINUX` to the enum and startup helper. Existing UI classes and `DeliveryApplication` remain completely unchanged.
3. **Adding a new UI Product Type (e.g. TextField):** Add `createTextField()` to `GUIFactory` interface. Implement `WindowsTextField` and `MacOSTextField`. Update `WindowsFactory` and `MacOSFactory` to implement the new method, and update `DeliveryApplication` to render the text field. This shows the main trade-off of Abstract Factory: adding new families is easy, but adding new product types requires changing the interface and all concrete factories.

6. References

1. Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). O'Reilly Media. Chapter 4: Factory Pattern.
2. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. Chapter 6: Objects and Data Structures.
3. Bekenov, Y. (2026). *Software Design Patterns - Lecture 2: Factory Method and Abstract Factory*. Astana IT University.